

Bulk RNA-seq Tasks

Anjana S Kumar

2026-06-09

R Markdown

Section 1: Raw FASTQ files processing

Q1.1 FASTQ vs FASTA

The major difference between FASTQ and FASTA is the content which builds them. FASTA is a standard text based format used to represent nucleotides and protein sequence using letter codes. They only have 2 elements in the format, beginning with ">" then Sequence ID + nucleotide/protein sequence. This format makes it suitable for reference genomes and protein databases.

On the other hand FASTQ format is used to store raw DNA or RNA sequences with their corresponding base-calling quality scores. Unlike FASTA these have 4 elements in the format, beginning with "@" + Seq ID + nucleotide sequence + quality scores in ASCII characters. Key difference is the Quality score where each base call is assigned a Phred score (Q) representing the probability of an incorrect base call: $Q = -10 \times \log_{10}(P)$.

Q1.2

Sequencing machines can make mistakes and so it is very important to check the quality of the sequence produced. To confirm data integrity, I would first verify file completeness using file size, does the size of the file I got matches the size reported from sequencing center. And then go forward with checksum validation, the generated checksum should exactly match the checksum provided by the sequencing center. Matching checksums confirm that the file has not been altered during transfer. Gzip integrity test can also be done to be sure as it checks whether a compressed .gz file is damaged or incomplete. A successful `gzip -t` check confirms that the file can be read and decompressed correctly. On bash these would be the commands to check -

- `md5sum file.fastq.gz` — generates checksum
- `gzip -t file.fastq.gz` — tests compression integrity
- `zcat file.fastq.gz | wc -l` — counts total lines (divides by 4 = reads)

Q1.3

After checking the data integrity it is important to assess sequencing quality using Phred scores and FastQC reports to identify low-quality reads, adapter contamination, abnormal GC content, or other sequencing artifacts before proceeding with analysis.

The FastQC Summary section gives a quick overview using: Green checkmark - Pass - Metric looks acceptable and No obvious quality issue. Yellow exclamation mark - Warning - Something unusual was detected maybe not necessarily a problem. Red Cross - Fail - Strong evidence of a quality issue that may require correction or filtering.

We have 3 modules which failed and 3 modules indicate warning

1. Per Tile Sequence Quality (FAIL) - This module failed because some sequencing tiles show lower quality than others. These lower-quality reads could be due to machine issue, imaging issue or flow cell issue.
2. Per Base Sequence Content (FAIL) - In this module we are checking if every position roughly has expected proportions of A/T/G/C or not. In the plot we can see G shoots up dramatically while other 3 bases reduce. This suggests: sequence composition bias, adapter contamination, polyG artifact. This made me instantly look at adapter content next.
3. Adapter Content (FAIL) (Most Important Here) - Here we are trying to find if the adapters are still present in the reads. This is the biggest issue in this report. Adapter content here increases dramatically after approximately 30 bp and becomes very high at the read end. This needs to be trimmed before alignment step.
4. Per Sequence GC Content (WARN) - The observed GC distribution shown in red line deviates from the theoretical distribution shown in blue line. This may indicate sequence bias, contamination, or enrichment of specific transcripts. But still the deviation is not severe.
5. Sequence Duplication Levels (WARN) - How many reads are exact copies of other reads is what we are looking for. For RNA seq high duplication is often expected because highly expressed genes produce many reads. After deduplication 60% reads remain, which is not alarming.
6. Overrepresented Sequences (WARN) - The report shows long T stretches and TruSeq adapter sequence, which also points towards adapter contamination and likely poly(A) read-through.

Q1.4 Strategies to Address QC Issues

Since the main issue we found was the adapters being present in the many read especially towards the end. It requires to be trimmed before alignment. Tools we can use for this: Cutadapt, Trimmomatic or Fastp. Cutadapt gives us control over the adapters manually and target the ones we need to eliminate. Trimmomatic - They are run in java, is built as a step-by-step pipeline optimized for standard Illumina data. Fastp which auto-detects adapters, eliminating the need to provide them manually.

These methods eliminate low quality bases too which improves alignment accuracy and reduces false-positive variant calls.

Another issue we noticed was uneven A/T/G/C proportions across read positions. This can be either due to adapter contamination which is already addressed. Or due to RNA-seq library preparation bias which is often reported as a protocol-specific artifact.

GC content's slight deviation could also be first addressed by trimming and then follow up if the contamination persists.

As for the sequence duplication levels, it's often expected because highly expressed genes generate many reads. Usually no action required. Per Tile Sequence Quality failures may indicate localized flow-cell or imaging issues. If necessary, reads coming from poor-quality tiles can be filtered using tools like fastp (–tile_based_filtering). But, this step is used when the tile-specific quality problems are severe, as mild tile effects often have little impact when overall read quality remains high.

Q1.5 Issue Affecting Alignment Rates

The most obvious issue that would affect alignment is the high adapter content, which failed in the FastQC report. Adapter sequences are synthetic oligonucleotide added during library preparation and are not part of the biological RNA sequence. If they are not removed before alignment, portions of reads will not match the reference genome, leading to lower alignment rates and an increase in mismatches. This can negatively affect downstream

analyses such as gene quantification and variant calling as they do not correspond to any real genomic sequence and will fail to align. Therefore, adapter trimming using tools such as Cutadapt, Trimmomatic, or fastp should be performed prior to alignment.

Q1.6 Trimmomatic vs Fastp

These are two of the techniques I mentioned in Q1.4 for adapter trimming. They are significantly different in speed, functionality and design.

Feature	fastp	Trimmomatic
Language used	C++	Java
Speed	Ultra-fast, data is read only once	Slower (sequential processing)
Adapter Detection	Automatic, no file needed	Manual, we need to provide adapter FASTA file
Visual Reports	Built-in HTML + JSON reports	Text logs only
Base Correction	Yes, it fixes mismatched bases in overlapping paired reads	It cannot fix bases
Poly-G Trimming	Yes, critical for NovaSeq/NextSeq data	No
Output Files (PE)	2 clean files	4 files (splits paired and unpaired reads)
Ease of Use	Beginner friendly, needs very little input because its defaults work for most data	Requires more configuration, requires a long string of specific settings and files
Customization	Good	Excellent, fine-grained step control

For our dataset, fastp would be much more helpful given the fact that we have poly G issue (PolyG content reaching 75% at read ends) to be addressed hence fastp can be used effectively here. whereas Trimmomatic has no equivalent functionality.

Q1.7 Trimmomatic Command and Parameters

Trimmomatic basically takes raw dirty FASTQ reads and cleans them up step by step like a pipeline of filters.

Basic structure looks like :

```
trimmomatic PE
```

```
[input files]
```

```
[output files]
```

```
[processing steps]
```

```

trimmomatic PE
input_R1.fastq.gz input_R2.fastq.gz
output_R1_paired.fastq.gz output_R1_unpaired.fastq.gz
output_R2_paired.fastq.gz output_R2_unpaired.fastq.gz
ILLUMINACLIP:TruSeq3-PE.fa:2:30:10
MINLEN:36

```

Parameter	Meaning
PE	Paired-end mode, two files (R1 and R2) that go together
input_R1/R2.fastq.gz	Input read files (forward and reverse)
output_R1/R2_paired.fastq.gz	Output files for reads where BOTH survived
output_R1/R2_unpaired.fastq.gz	Output files for reads where only ONE survived
ILLUMINACLIP:TruSeq3-PE.fa:2:30:10	Remove TruSeq adapters. 2=seed mismatches allowed, 30=palindrome clip threshold, 10=simple clip threshold
MINLEN:36	Drop reads shorter than 36bp after trimming

Traditionally these 3 commands are also used

LEADING:3 - Remove bases from start of read if quality < 3

TRAILING:3 - Remove bases from end of read if quality < 3

SLIDINGWINDOW:4:15 - Scan with 4bp window, cut when average quality < 15

Modern consideration:

It is worth noting that aggressive quality trimming (LEADING, TRAILING, SLIDINGWINDOW) which traditionally used to be included is increasingly debated in modern pipelines. Many bioinformaticians now prefer a lighter approach keeping only adapter removal and minimum length filtering, as modern aligners like STAR handle low-quality bases through soft-clipping. For this dataset, given the severe adapter contamination identified in Q1.3, ILLUMINACLIP and MINLEN:36 are the critical steps.

Section 2

Q2.1 RNA-seq Aligner Comparison

A major challenge for RNA-seq aligners is to be splice aware. Because RNA-seq data comes from mRNA which is intron free, this is aligned with reference genome which still has the introns. Hence taking in account of this is really important.

Feature	TopHat2	STAR	HISAT2	Subread
Speed	Slow	Very fast	Fast	Fast
Memory	Moderate (~8GB)	High (~32GB -40GB)	Low (~4GB)	Low

Feature	TopHat2	STAR	HISAT2	Subread
Accuracy	Moderate	High	High	Moderate
Splice junction detection	Good	Excellent	Good	Basic
Algorithm	Bowtie2 based	Suffix array	Graph-based hierarchical index	Seed-and-vote
Status	Outdated (replaced by HISAT2)	Gold standard	Actively used	Better for DNA-seq
Used by	Legacy pipelines	ENCODE, GTEx	Memory-limited environments	featureCounts counting
Novel junction detection	Limited	Excellent	Good	Limited

Q2.2 — Exon-Intron Junction Strategies

While aligning the reference genome to the RNA Seq data some reads span across splice junctions. Like they may start from exon 1 and end at exon 2 with the intron in between of the reference genome. this is something the aligner need to resolve. Each aligner have a different approach for this. I have summarised their working here :

TopHat2: TopHat2 tackles this in two rounds. First it aligns all the reads that map easily (the continuous reads). Then it goes to the leftover reads that don't map, it splits these reads to smaller segments and each segment is mapped separately. Example - The read from mRNA spans exon1-exon2 boundary. It fails continuous alignment because the intron between them is too large. When split, each segment aligns to its respective exon on the reference genome, revealing the intron gap as a splice junction. It uses known GT-AG splice site rules to guide this process (GT-AG are the canonical sequences at intron boundaries in most organisms).

STAR: STAR uses a clever approach called Maximal Mappable Prefix (MMP). STAR clips the first MMP, and when the read stops matching the genome (because of an intron) it breaks there, then takes the remaining piece and finds the next longest match and this is repeated until the whole read is placed and stitching everything together across the junction at the end. This is why STAR is so fast and accurate at detecting both known and novel junctions.

HISAT2: HISAT2 uses a two-level indexing strategy which works one after the other. It has one large global index for the whole genome and thousands of small local indices covering specific genomic regions. When a read spans a junction, the global index gives it a rough idea of the location and then the local index does the precise fine-grained alignment. This hierarchical approach is what makes HISAT2 much more memory efficient than STAR while still being accurate at junctions.

Subread/Subjunc: Subread breaks each read into several short 16bp pieces called seeds. Each short seed is mapped independently to the genome and these seeds "vote" on where the read should go. For junction reads specifically, the companion tool Subjunc uses these votes to figure out where the exon-exon boundary is. So if a read spans an exon-exon junction, the seeds from the front of the read will vote for Exon A, and the seeds from the back of the read will vote for Exon B. If seeds from the same read vote for two different genomic locations, Subjunc identifies the gap between them as a splice junction.

Q 2.3 - Coverting json file to R dataframe and .txt

```
knitr::opts_chunk$set(echo = TRUE)
library(jsonlite)
json_data <- fromJSON("/Users/ask/Downloads/data_RNASeq/demo_alignQC_flagStat.json")  #
Read the json file
str(json_data)
```

```

## List of 2
## $ QC-passed reads:List of 20
## ..$ total : int 117346035
## ..$ primary : int 69616187
## ..$ secondary : int 47729848
## ..$ supplementary : int 0
## ..$ duplicates : int 0
## ..$ primary duplicates : int 0
## ..$ mapped : int 92298651
## ..$ mapped % : num 78.7
## ..$ primary mapped : int 44568803
## ..$ primary mapped % : num 64
## ..$ paired in sequencing : int 68922056
## ..$ read1 : int 34461028
## ..$ read2 : int 34461028
## ..$ properly paired : int 42720516
## ..$ properly paired % : num 62
## ..$ with itself and mate mapped : int 43183008
## ..$ singletons : int 1065924
## ..$ singletons % : num 1.55
## ..$ with mate mapped to a different chr : int 55764
## ..$ with mate mapped to a different chr (mapQ >= 5): int 35171
## $ QC-failed reads:List of 20
## ..$ total : int 0
## ..$ primary : int 0
## ..$ secondary : int 0
## ..$ supplementary : int 0
## ..$ duplicates : int 0
## ..$ primary duplicates : int 0
## ..$ mapped : int 0
## ..$ mapped % : NULL
## ..$ primary mapped : int 0
## ..$ primary mapped % : NULL
## ..$ paired in sequencing : int 0
## ..$ read1 : int 0
## ..$ read2 : int 0
## ..$ properly paired : int 0
## ..$ properly paired % : NULL
## ..$ with itself and mate mapped : int 0
## ..$ singletons : int 0
## ..$ singletons % : NULL
## ..$ with mate mapped to a different chr : int 0
## ..$ with mate mapped to a different chr (mapQ >= 5): int 0

```

```
print(json_data)
```

```
## `$QC-passed reads`  
## `$QC-passed reads`$total  
## [1] 117346035  
##  
## `$QC-passed reads`$primary  
## [1] 69616187  
##  
## `$QC-passed reads`$secondary  
## [1] 47729848  
##  
## `$QC-passed reads`$supplementary  
## [1] 0  
##  
## `$QC-passed reads`$duplicates  
## [1] 0  
##  
## `$QC-passed reads`$`primary duplicates`  
## [1] 0  
##  
## `$QC-passed reads`$mapped  
## [1] 92298651  
##  
## `$QC-passed reads`$`mapped %`  
## [1] 78.66  
##  
## `$QC-passed reads`$`primary mapped`  
## [1] 44568803  
##  
## `$QC-passed reads`$`primary mapped %`  
## [1] 64.02  
##  
## `$QC-passed reads`$`paired in sequencing`  
## [1] 68922056  
##  
## `$QC-passed reads`$read1  
## [1] 34461028  
##  
## `$QC-passed reads`$read2  
## [1] 34461028  
##  
## `$QC-passed reads`$`properly paired`  
## [1] 42720516  
##  
## `$QC-passed reads`$`properly paired %`  
## [1] 61.98  
##  
## `$QC-passed reads`$`with itself and mate mapped`  
## [1] 43183008  
##  
## `$QC-passed reads`$singletons  
## [1] 1065924  
##
```



```
## $`QC-passed reads`$`singletons` %`  
## [1] 1.55  
##  
## $`QC-passed reads`$`with mate mapped to a different chr`  
## [1] 55764  
##  
## $`QC-passed reads`$`with mate mapped to a different chr (mapQ >= 5)`  
## [1] 35171  
##  
##  
## $`QC-failed reads`  
## $`QC-failed reads`$total  
## [1] 0  
##  
## $`QC-failed reads`$primary  
## [1] 0  
##  
## $`QC-failed reads`$secondary  
## [1] 0  
##  
## $`QC-failed reads`$supplementary  
## [1] 0  
##  
## $`QC-failed reads`$duplicates  
## [1] 0  
##  
## $`QC-failed reads`$`primary duplicates`  
## [1] 0  
##  
## $`QC-failed reads`$mapped  
## [1] 0  
##  
## $`QC-failed reads`$`mapped` %`  
## NULL  
##  
## $`QC-failed reads`$`primary mapped`  
## [1] 0  
##  
## $`QC-failed reads`$`primary mapped` %`  
## NULL  
##  
## $`QC-failed reads`$`paired in sequencing`  
## [1] 0  
##  
## $`QC-failed reads`$read1  
## [1] 0  
##  
## $`QC-failed reads`$read2  
## [1] 0  
##  
## $`QC-failed reads`$`properly paired`  
## [1] 0
```

```
##  
## $`QC-failed reads`$`properly paired %`  
## NULL  
##  
## $`QC-failed reads`$`with itself and mate mapped`  
## [1] 0  
##  
## $`QC-failed reads`$singletons  
## [1] 0  
##  
## $`QC-failed reads`$`singletons %`  
## NULL  
##  
## $`QC-failed reads`$`with mate mapped to a different chr`  
## [1] 0  
##  
## $`QC-failed reads`$`with mate mapped to a different chr (mapQ >= 5)`  
## [1] 0
```

```
# extracting the QC passed and failed data  
qc_passed <- json_data$`QC-passed reads`  
qc_failed <- json_data$`QC-failed reads`  
# Convert NULL values to NA first to avoid errors or misalignment  
qc_passed <- lapply(qc_passed, function(x) if(is.null(x)) NA else x)  
qc_failed <- lapply(qc_failed, function(x) if(is.null(x)) NA else x)  
# Convert to dataframe  
df <- data.frame(  
  Metric = names(qc_passed),  
  QC_passed = unlist(qc_passed),  
  QC_failed = unlist(qc_failed),  
  row.names = NULL  
)  
print(df)
```

##	Metric	QC_passed	QC_failed
## 1	total	117346035.00	0
## 2	primary	69616187.00	0
## 3	secondary	47729848.00	0
## 4	supplementary	0.00	0
## 5	duplicates	0.00	0
## 6	primary duplicates	0.00	0
## 7	mapped	92298651.00	0
## 8	mapped %	78.66	NA
## 9	primary mapped	44568803.00	0
## 10	primary mapped %	64.02	NA
## 11	paired in sequencing	68922056.00	0
## 12	read1	34461028.00	0
## 13	read2	34461028.00	0
## 14	properly paired	42720516.00	0
## 15	properly paired %	61.98	NA
## 16	with itself and mate mapped	43183008.00	0
## 17	singletons	1065924.00	0
## 18	singletons %	1.55	NA
## 19	with mate mapped to a different chr	55764.00	0
## 20	with mate mapped to a different chr (mapQ >= 5)	35171.00	0

```
cat("Dimensions:", dim(df), "\n")
```

```
## Dimensions: 20 3
```

```
# Write to txt file
write.table(df,
            file = "alignQC_flagstat.txt",
            sep = "\t",
            row.names = FALSE,
            quote = FALSE)

cat("Text file is saved!\n")
```

```
## Text file is saved!
```

The relatively low mapping rate (78.66%) is maybe due to adapter contamination identified in Q1.3. After adapter trimming, alignment rates are expected to improve.

Q2.4 - Salmon and Kallisto Rationale

Kallisto Kallisto builds a continuous index map of sequences. When it reads a sequence, it checks a few positions which acts as key phrases. If those positions already confirm the exact gene target, Kallisto physically skips the rest of the sequence. It basically just looks for key words in the reads which directs it to the transcript which includes that key word rather than looking base by base. It is the best choice when we want our results computed immediately and have limited computer power.

Salmon Salmon does a similar fast lookup, but it keeps an eye out for patterns of bias which is unlike kallisto. It notices if our sequencing machine accidentally over-copied sequences with certain chemical traits (like high GC content) or chopped the fragments unevenly. It builds a mathematical model of these glitches and subtracts that background noise from our final gene counts.

Q 2.5 conventinal packages VS pseudoalignment packages

Q2.5 — Pseudoalignment vs Conventional Aligners

Feature	Pseudoalignment (like Salmon/Kallisto)	Conventional Alignment (like STAR/HISAT2)
Question asked	Which transcript?	Which spot precisely in genome?
How they think	They treat reads like votes for a specific gene. They break reads into key phrases, see which transcript lists contain those phrases, group ambiguous reads into bins, and let statistical math (the EM algorithm) distribute the counts perfectly at the very end	They care deeply about geography and coordinates. They want to find the exact chromosome, the exact start position, and the exact base pair where an intron begins and ends
Speed	Ultra-fast (minutes)	Slower (hours for large datasets)
Memory	Low	High (STAR ~30GB)
The Goal	Count gene activity as fast as possible	Map the physical layout of the genome
Splice junction detection	No — works on transcriptome	Yes — genome-based
Novel transcript detection	No	Yes
Bias correction	Yes (Salmon)	Limited
Alignment file	Not produced	BAM file produced
Use case	Gene/transcript quantification	Quantification + variant calling + novel junction discovery
Accuracy	Comparable for expression	Higher for genomic analyses

Section 3

Q 3.1 featureCounts column explanation

Geneid : Is the Ensembl gene Identifier, which gene the row represents. Chr : Which chromosome is the gene located on Start : Starting genomic coordinate of the gene End : Ending genomic coordinate of the gene Strand : DNA strand on which the gene is located, the +/- denotes the orientation of strand reverse (-) or forward (+). Length : Length of the gene feature being counted in base pairs. Eg - 2541, means the gene spans 2541 bp. counts : Number of reads mapped to this gene in the sample (the BAM file path is the column header). A count of 0 means no reads mapped to that gene in this sample — the gene was either not expressed or expressed below detection threshold. For example, ENSG00000290825 has a count of 0 while ENSG00000238009 has 102 reads mapped.

Q 3.2 Basic QC Summary

```
counts <- read.table("/Users/ask/Downloads/data_RNASeq/demo_count.txt",
                    header = TRUE,
                    skip = 1,
                    sep = "\t")
colnames(counts)[7] <- "counts"

cat("=== Basic Dataset Info ===\n")
```

```
## === Basic Dataset Info ===
```

```
cat("Total genes in annotation:", nrow(counts), "\n")
```

```
## Total genes in annotation: 38582
```

```
cat("Genes with at least 1 count:", sum(counts$counts > 0), "\n")
```

```
## Genes with at least 1 count: 6186
```

```
cat("Genes with zero counts:", sum(counts$counts == 0), "\n")
```

```
## Genes with zero counts: 32396
```

```
cat("Detection rate:", round(sum(counts$counts > 0)/nrow(counts)*100, 1), "%\n")
```

```
## Detection rate: 16 %
```

```
cat("\n=== Count Distribution ===\n")
```

```
##
## === Count Distribution ===
```

```
summary(counts$counts)
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
##      0.0      0.0      0.0    223.8     0.0 1493327.0
```

```
cat("\n=== Library Size ===\n")
```

```
##
## === Library Size ===
```

```
cat("Total mapped reads:", sum(counts$counts), "\n")
```

```
## Total mapped reads: 8635319
```

```
cat("\nGenes with counts > 10:", sum(counts$counts > 10), "\n")
```

```
##
## Genes with counts > 10: 4447
```

```
cat("Genes with counts > 100:", sum(counts$counts > 100), "\n")
```

```
## Genes with counts > 100: 3912
```

```
cat("\nTop 5 expressed genes:\n")
```

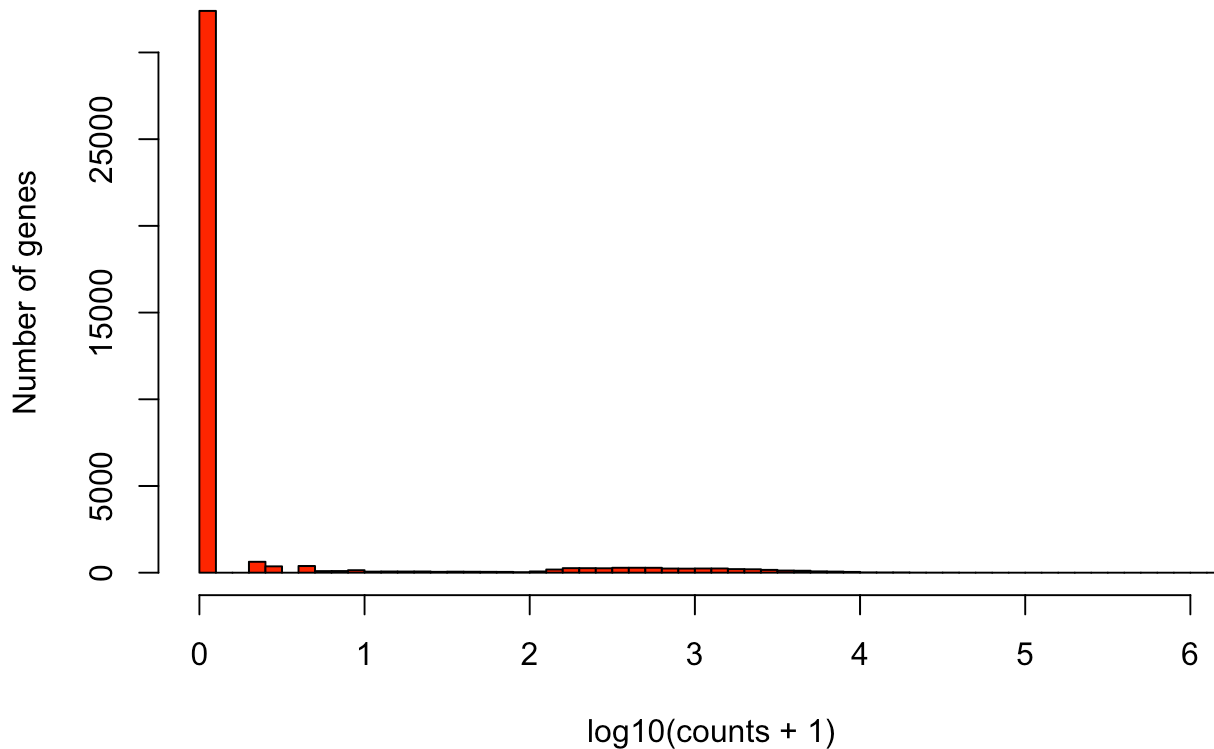
```
##
## Top 5 expressed genes:
```

```
head(counts[order(counts$counts, decreasing = TRUE),
               c("Geneid", "counts")], 5)
```

```
##           Geneid  counts
## 21163 ENSG00000251562 1493327
## 35719 ENSG00000280441  510454
## 10527 ENSG00000131711 1071104
## 23316 ENSG00000256259   60095
## 15910 ENSG00000104722   58954
```

```
# Distribution plot
hist(log10(counts$counts + 1),
     main = "Distribution of log10(counts + 1)",
     xlab = "log10(counts + 1)",
     ylab = "Number of genes",
     col = "red",
     breaks = 50)
```

Distribution of log10(counts + 1)



Q3.3 – Normalization Methods

Why do we need normalization? For multiple sample normalization is a very important step to eliminate biases. library size differences (different sequencing depths) and gene length differences (longer genes attract more reads).

Method	Corrects Library Size	Corrects Gene Length	Cross-sample Comparable	Best Use
CPM	Yes	No	Yes	Comparing same gene across samples

Method	Corrects Library Size	Corrects Gene Length	Cross-sample Comparable	Best Use
RPKM	Yes	Yes	No	same sample gene comparison (single-end)
FPKM	Yes	Yes	No	same sample gene comparison (paired-end)
TPM	Yes	Yes	Yes	Current standard , comparable cross-sample and cross-gene

Why TPM is preferred over RPKM/FPKM:

RPKM/FPKM normalize by library size first then gene length. This means the sum of all RPKM/FPKM values differs between samples, making direct comparison between samples unreliable.

TPM normalizes by gene length first then library size. This ensures all samples sum to exactly 1,000,000 — making TPM values directly comparable between samples.

It is the order of operations which makes massive difference between them.

Q3.4 — Outlier Detection Strategies

When we are working with multiple samples after normalization, identifying outliers early is critical before running differential expression analysis.

My primary approach — PCA: PCA is my first go-to for outlier detection. It reduces the entire expression profile of each sample into two dimensions, making it easy to spot samples that behave differently from their group. Like if 3 controls don't sit together that could potentially be an outlier. we can also check if these outliers are samples processed on same day indicating batch effects. Samples from the same condition should cluster together. I find this the most intuitive approach because you can see the overall structure of your data at a glance.

My second check — Boxplots and Density Plots: After PCA, I would look at boxplots and density plots. In a well-normalized dataset, all samples should have similar distributions with comparable medians and spread. An outlier sample will show a visibly different distribution like shifted median, different spread, or a different shape in the density plot. These help in catching outliers that might not be obvious in PCA but have systematic expression differences across many genes.

Other useful approaches: - **Sample-to-sample correlation heatmap** — calculates pairwise correlations between samples. An outlier shows low correlation with all other samples. - **Hierarchical clustering** — This is again a visual method to find outliers. A sample clustering with the wrong group suggests a sample swap. - **Library size check** — a sample with dramatically fewer total counts likely failed during preparation.

Q 3.5 - DESeq2 vs limma vs edgeR

Feature	edgeR	DESeq2	limma-voom
Statistical model	Negative binomial GLM	Negative binomial with shrinkage	Linear model on log-CPM (voom)
Data Input	Raw integer counts	Raw integer counts	Normalized continuous values (logCPM)
Normalization	TMM (Trimmed mean of M values)	Median of ratios	TMM or quantile
Fold change shrinkage	No	Yes (apeglm/ashr)	No
Design flexibility	Excellent	Good	Excellent
Speed	Fast	Moderate	Very fast
Sample size	Small-medium	Small	Medium-large
Custom contrasts	Excellent	Limited	Excellent

edgeR

edgeR is my preferred tool for pseudobulk differential expression in single-cell RNA-seq data where counts are aggregated per cell type per sample before applying the bulk RNA-seq GLM framework. In my thesis work on Huntington's disease snRNA-seq data, I used edgeR's GLM with explicit contrast matrices across multiple cell types, demonstrating its flexibility for complex multi-contrast designs. Limitations: It handles outliers slightly less aggressively by default than DESeq2, meaning highly noisy datasets require a bit more manual filtering and quality control up front. Best Use Case: Excellent for medium-scale datasets or complex, where we need multi-layered experimental matrices which has highly customizable GLM testing.

DESeq2:

DESeq2 also uses negative binomial modelling but its key strength is fold change shrinkage it pulls extreme fold changes toward zero for genes with low counts or high variability. This makes results more conservative and reliable, particularly valuable for small sample sizes (<6 per group). Limitation: less flexible for complex multi-contrast designs compared to edgeR. And is computationally heavy. Best Use Case: The default choice for standard bulk RNA-seq experiments with small sample sizes (e.g., 3 Controls vs. 3 Treated).

limma-voom:

Originally developed for microarray data and extended to RNA-seq through the voom transformation, which converts counts to log-CPM values with precision weights. Extremely fast and flexible for complex designs. Best suited for larger datasets (>20 samples per group) where the normal approximation becomes more reliable. Limitation: voom transformation adds an extra step and may not model count data as naturally as edgeR or DESeq2. Also, it struggles with very small sample sizes. Best Use Case: An absolute gold standard for large-scale transcriptomic cohorts (e.g., human clinical data) and complex batch-correction designs.

Section 4

Q 4.1 FEA and GSEA

Functional Enrichment Analysis (FEA) - It operates on a discrete, threshold-based statistical framework using the hypergeometric distribution. We need to first apply arbitrary cutoffs (such as p value < 0.05 and $\log_2$ fold change > 1) to split the entire transcriptome into two distinct categories: a “significant” list and a “background” universe. FEA then checks each pathway and asks Does this pathway contain more significant genes than we would expect by random chance?.

Gene Set Enrichment Analysis (GSEA) - Unlike FEA, GSEA does not use a significance cutoff such as $p < 0.05$. Instead, it uses all genes in the dataset. It looks if the genes in the dataset from a particular pathway tend to appear near bottom or top of its ranked list. It ranks the genes from top to bottom - top being upregulated, bottom being downregulated and the genes which fall in middle most probably unchanged. GSEA ranks all genes and tests whether genes from a pathway are consistently found near the top or bottom of the ranked list, avoiding the need for arbitrary significance cutoffs.

Q 4.2 PROS and CONS

Functional Enrichment Analysis (FEA) / ORA Pros: Extremely simple math (Fisher’s Exact Test) that finishes in seconds. Focuses exclusively on the strongest, highest-magnitude biological signals by filtering out everything else. Low Input Requirement: Just requires a simple, unordered list of gene names. Cons: FEA depends heavily on the chosen cutoff. Completely ignores a gene with a p -value of 0.051 while celebrating one at 0.049. Treats a master transcription factor with a massive 100-fold change identically to a minor downstream gene with a 1.2-fold change. Blind to entire signaling cascades or metabolic pathways where dozens of genes shift in unison by small, coordinated amounts (making a huge difference).

Gene Set Enrichment Analysis (GSEA) Pros: Uses the entire continuous transcriptome, eliminating selection bias. Successfully identifies pathways where genes shift by subtle, consistent amounts that would fail individual threshold tests. Cons: Computationally expensive, especially on single-cell scales. Cannot use a partial or pre-filtered candidate list it strictly requires data for the entire genome to work. A pathway can score high purely due to a huge block of mildly changed genes, making it harder to pick a single clear target for lab validation.

Q 4.3 rMATS detecting splicing events

rMATS detects five major types of alternative splicing events: Skipped Exons (SE), Retained Introns (RI), Alternative 5’ Splice Sites (A5SS), Alternative 3’ Splice Sites (A3SS), and Mutually Exclusive Exons (MXE).

Differential splicing is assessed by comparing the inclusion levels (IncLevel1 and IncLevel2) between two groups in rMATS. The interpretation of “inclusion” depends on the event type:

SE (Skipped Exon): Inclusion means the exon is retained in the mature transcript rather than skipped. RI (Retained Intron): Inclusion in this case means the intron remains in the mature transcript instead of being removed during splicing. A5SS (Alternative 5’ Splice Site): Usage of a different 5’ splice site. Changing the start of exon, resulting in different exon lengths. A3SS (Alternative 3’ Splice Site): Inclusion reflects usage of one 3’ splice site over an alternative site. Changing the end of an exon. MXE (Mutually Exclusive Exons): preferential usage of one mutually exclusive exon over the alternative exon.

The IncLevelDifference represents the difference in event usage between the two groups (like between control and condition), while the FDR indicates statistical significance after multiple-testing correction. A significant FDR together with a large inclusion difference suggests a biologically meaningful change in splicing between conditions.

For example, in an RI event, a higher inclusion level indicates increased intron retention. In an MXE event, a higher inclusion level indicates increased usage of one particular exon to its mutually exclusive counterpart.

Q 4.4 What if splice variants are novel

If a splice variant is not in the reference GTF, tools like featureCounts or rMATS cannot fully identify it because they rely on existing annotations. We need to start with aligning the reads first here using a splice-aware aligner such as STAR or HISAT2 to detect known and novel splice junctions. I would then perform transcript assembly using StringTie or Cufflinks to reconstruct transcript isoforms directly from the aligned reads. Assemblies from multiple samples can be merged and compared with the reference annotation using gffcompare to identify novel transcripts, exons, and splice junctions.

Q 4.5 how to perform differential transcript usage

To perform Differential Transcript Usage (DTU) analysis incorporating the newly reconstructed master GTF, the following four-step pipeline would be executed:

Using a new GTF, we need to first count the individual transcript using tools such as Salmon, Kallisto, or StringTie and then make count matrix for each sample. Through this we can know if there is difference in transcript usage between two groups. Even if the total expression stays same, transcript contribution to expression can be different. The resulting transcript count matrix we made would then be analyzed using DTU methods such as DRIMSeq, DEXSeq, or IsoformSwitchAnalyzeR. These methods compare the relative usage of transcript isoforms between conditions rather than total gene expression. Significant changes in transcript proportions indicate differential transcript usage and may reveal isoform switching events that are not detected by gene-level differential expression analysis.